

DOCUMENTO QUE ACOMPANHA A 2ª ENTREGA DO PROJETO DE SD

1) Quais dos seguintes requisitos foram corretamente resolvidos?

Abaixo de cada requisito, respondam: "Sim" ou "Não" ou "Sim mas com erros: [enumerar sucintamente os erros]"

- Estender o sistema para permitir nós replicados

Sim

- Difusão baseada em blocos (e não transações individuais)

Sim

- Variantes não-bloqueantes dos comandos

Sim

- Funcionalidade de atrasar, no nó, a execução de cada pedido

Sim

- Suporte ao lançamento de novos nós a qualquer ponto no tempo

Sim

- Tolerância a falhas silenciosas dos nós

Sim

2) Descrevam sucintamente as principais alterações que o grupo aplicou a cada componente indicada abaixo (máx. 800 palavras)

Idealmente, componham uma lista de itens (cada um iniciado por um hífen, tal como a listagem dos requisitos apresentada acima).

Refiram ainda se houve alterações aos argumentos de linha de comando dos executáveis, quais foram e qual a sua justificação.

Aos .proto:

- Adicionámos a variável `commandNumber` aos pedidos do cliente de modo a permitir que os mesmos tenham um identificador único. Isto permite, por sua vez, que os mesmos não sejam tratados múltiplas vezes por diferentes nós;

- Criámos `Transaction`, que representa uma transação, podendo ser do tipo `CreateWallet`, `DeleteWallet` ou `Transfer` e contendo também o `commandNumber`, e a resposta correspondente `TransactionResponse`;

- Criámos `DeliverBlockRequest` e `DeliverBlockResponse`, invocadas quando um nó pede um bloco ao sequenciador;

- Criámos `PullBlockchainRequest` e `PullBlockchainResponse` de modo a permitir que um nó seja inicializado com o estado da blockchain no seu momento de criação e no atendimento de pedidos deste tipo.

Ao programa do Cliente:

- Criámos a classe `TransactionObserver` para implementar as nossas próprias funções do `StreamObserver`, o que foi importante para implementar corretamente as variantes não-bloqueantes dos comandos;

- Implementámos uma função geral para percorrer os vários nós de confiança e executar a transação, que depois encaminha para um dos três casos particulares `create`, `delete` ou `transfer`;

- Adaptámos todas as funções necessárias para trabalharem com a versão geral das mensagens (o `Transaction` e `TransactionResponse`) e com as variantes não-bloqueantes dos comandos;

- Adicionámos a função auxiliar `generateCommandNumber` para gerar o identificador de transação.

Ao programa do Nó:

- Adicionámos a função `executeTransaction()` que recebe uma transação do cliente, encaminha-a para o Sequenciador e depois atualiza a sua blockchain local de acordo com a do Sequenciador até chegar ao bloco da transação recebida;
- Adicionámos a função `parseTransaction()` que recebendo uma transação encaminha para um dos seus três casos particulares e aplica as mudanças à blockchain local do nó;
- Implementámos a função `pullBlockchain()` que corre sempre que um nó é lançado e faz uma cópia da blockchain do Sequenciador para a blockchain local do nó;
- Adicionámos a classe `DelayIntercetptor`, que passou a ser chamada sempre que o serviço do nó é criado, que intercepta as mensagens entre o cliente e o servidor e aplica o tempo de espera (delay) presente nos metadados;
- Implementámos a função `deliverBlock()` que pede um bloco ao Sequenciador e o retorna

Ao programa do Sequenciador:

- Quando o sequenciador recebe um broadcast, verifica se já existe uma transação com aquele `commandNumber` e, se sim, retorna -1, para indicar que a mesma já está a ser tratada por outro nó. Além disso, implementámos a lógica de verificar se um certo bloco já foi fechado e, se sim, criar outro bloco para colocar a transação;
- Implementámos a função `deliverBlock`, que envia o bloco pedido a um nó, após esperar que ele feche;
- Implementámos a função `pullBlockchain`, que envia ao nó o estado atual da blockchain como resposta.
- Criámos a classe `Block` que representa um bloco e contém a sua lógica de fecho, sendo gerida automaticamente por ele mesmo

- Não houve alterações aos argumentos de linha de comando dos executáveis

3) Na vossa solução, as transações recebidas pelo sequenciador levam algum identificador?

Se sim, expliquem brevemente o formato e como é gerado o identificador (máx. 100 palavras)

Sim. Quando um cliente faz um pedido, é gerado um identificador, multiplicando o seu pid por 10 e pelo número de dígitos do contador atual e adicionado o contador atual. Deste modo, identificamos as transações unicamente e evitamos o tratamento repetido das mesmas.

4) Como foi frisado na primeira aula teórica, não é aceitável o uso de GenAI/Code Copilots para gerar código usado diretamente no projeto.

Tendo isto em conta, respondam brevemente às seguintes 3 perguntas:

i) Os membros do grupo conseguem explicar o código submetido se tal for perguntado na discussão do projeto (feita sem recurso a AI)?

Sim

ii) Os membros do grupo conseguem defender todas as decisões de desenho e implementação se tal for perguntado na discussão do projeto (feita sem recurso a AI)?

Sim

iii) Usaram alguma(s) ferramenta(s) de AI? Se sim, para que uso?

Sim, usámos AI para ajudar no diagnóstico de erros e para discutir possíveis implementações, mas não para gerar código incluído no projeto automaticamente.